



THINKST

SCAPES

Q2 • 2023

<https://thinkst.com/ts>

Brought to you by



**Most companies find out way too late
that they've been breached.**

Thinkst Canary changes this.

Canaries deploy in under 4 minutes
and require 0 ongoing admin overhead.

They remain silent until they need to chirp,
and then, you receive that single alert.

When.it.matters.

Find out why some of the smartest security teams
in the world swear by Thinkst Canary.

<https://canary.love>

Contents

Introduction	4
Themes covered in this issue	5
Privacy in the modern era	6
IPvSeeYou: Exploiting Leaked Identifiers in IPv6 for Street-Level Geolocation	7
Device Tracking via Linux's New TCP Source Port Selection Algorithm	8
zk-creds: Flexible Anonymous Credentials from zkSNARKs and Existing Identity Infrastructure	9
3 Years in China: A Tale of Building a REAL Full Speed Anti-Censorship Router	10
Embedded [in]security	11
Embedded Threats: A Deep Dive into the Attack Surface and Security Implications of eSIM Technology	12
RPMB, a secret place inside the eMMC	13
Compromising Garmin's Sport Watches: A Deep Dive into GarminOS and its MonkeyC Virtual Machine	14
The Impostor Among US(B): Off-Path Injection Attacks on USB Communications	15
MagBackdoor: Beware of Your Loudspeaker as A Backdoor For Magnetic Injection Attacks	16
Issues at the operating system level	17
(Windows) Hello from the Other Side	18
Every Signature is Broken: On the Insecurity of Microsoft Office's OOXML Signatures	19
Dirty Bin Cache: A New Code Injection Poisoning Binary Translation Cache	20
The Most Dangerous Codec in the World: Finding and Exploiting Vulnerabilities in H.264 Decoders	21
Nifty Sundries	22
EverParse: Secure Binary Data Parsers for Everyone	23
InfinityGauntlet: Expose Smartphone Fingerprint Authentication to Brute-force Attack	24
It's (DOM) Clobbering Time: Attack Techniques, Prevalence, and Defenses	25
Can you trust ChatGPT's package recommendations?	26
Phoenix Domain Attack: Vulnerable Links in Domain Name Delegation and Revocation	27
Man-in-the-Middle Attacks without Rogue AP: When WPAs Meet ICMP Redirects	28
Conclusion	29

Cover photo: A spiderweb covered in waterdrops.

Introduction

Welcome to the Q2, 2023 edition of ThinkstScapes! This issue focuses on content released, published, or presented between the first of April and the end of June, 2023.

Quarter 2 showed a strong contingent of content presented at conferences, with a weakening of blog content. There appears to be an inverse relationship between talks/papers and blog posts: We suspect that in quarters with lots of top-tier conferences, more content is saved for the bigger talk reveals, whereas quarters with fewer big conferences see more blog posts being used for unscheduled research releases.

As a reminder: if you are aware of work we've missed, a blog post we should have seen or a conference we should have covered, we'd love to hear about it. Please send them to **ts@thinkst.com!**

In addition to over 1 300 blog posts, this quarter's content was drawn from the following conference presentations:

Conference name	Number of talks
Black Hat Asia	44
fwd:cloudsec	39
HITB AMS 2023	23
THCon 2023	16
LeHack Kernel Panic!	22
T2'23 INFOSEC CONFERENCE	10
STHACK	7
PasswordsCon	16
OffensiveCon	18
Positive Hack Days	26
THOTCON	30
x33fcon	24
Hack Miami	22
Hardwear.io NL	14
REcon	27
CircleCityCon	14
TyphoonCon	20
LangSec Workshop	13
IEEE Symposium on Security and Privacy	196
Workshop on Offensive Technologies	21
SSTIC	35
Troopers	35
USENIX Security Symposium	92
Total	764



Themes covered in this issue

1. PRIVACY IN THE MODERN ERA

Work in this theme looks at the relationships between security and privacy, showing concerning privacy leaks and promising new technologies. Look for IPv6 privacy leaks, tracking Linux devices through VPNs, a novel zero-knowledge authentication scheme, and how to bypass the Great Firewall of China.

2. EMBEDDED [IN]SECURITY

This theme looks deeper at embedded device security, as part of specific devices or as a component to larger systems. There's something here for everyone, with papers on eSIM weaknesses, hacking Garmin smartwatches, off-path USB attacks, and attacking voice-assistants using magnetic fields to inject audio. On a more positive note, it turns out there is a powerful primitive to be found in many embedded memory cards, that can bootstrap a more trustworthy experience.

3. ISSUES AT THE OPERATING SYSTEM LEVEL

Talks in this theme explore issues at the OS with far-reaching security consequences. Look for edge cases in Azure AD with Windows Hello, failing to validate signatures on Office documents, issues in translating applications to support the M1 and M2 processors, and issues with parsing videos on iOS.

4. NIFTY SUNDRIES

As always, there are some great talks that didn't fit into any of the other themes, but warranted inclusion. This issue highlights a deployed parser generation framework, a way to bypass fingerprint authentication limits on mobile devices, DOM clobbering attacks, ChatGPT hallucinations, methods to keep a sinkholed domain active, and a rogue AP attack without the rogue AP.

A cheetah standing on a large branch.

Privacy in the modern era

- ✓ *IPvSeeYou: Exploiting Leaked Identifiers in IPv6 for Street-Level Geolocation*
- ✓ *Device Tracking via Linux's New TCP Source Port Selection Algorithm*
- ✓ *zk-creds: Flexible Anonymous Credentials from zkSNARKs and Existing Identity Infrastructure*
- ✓ *3 Years in China: A Tale of Building a REAL Full Speed Anti-Censorship Router*



IPvSeeYou: Exploiting Leaked Identifiers in IPv6 for Street-Level Geolocation

Author: Erik Rye and Robert Beverly



[Citation](#)



[Slides](#)



[Paper](#)



[Code](#)

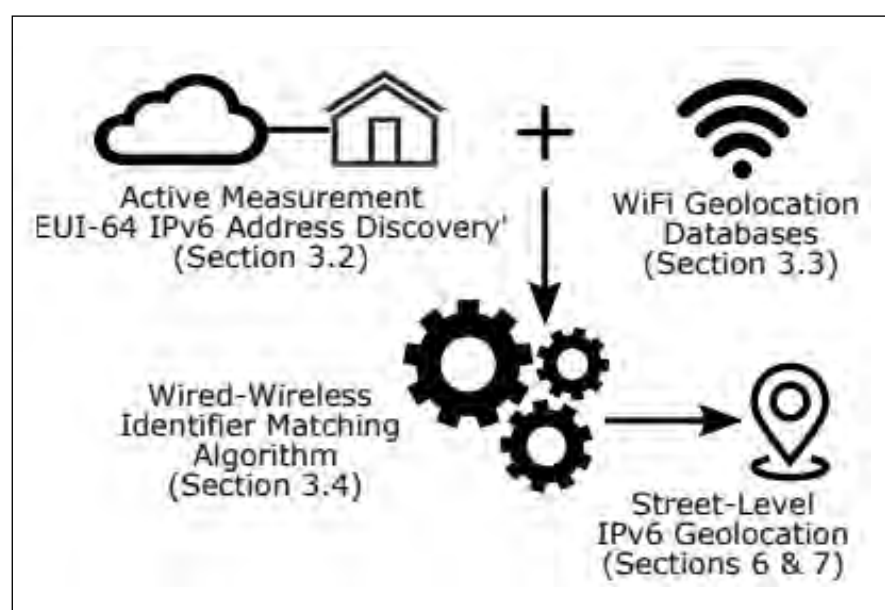
This research performed data correlation between legacy IPv6 networking devices (i.e. routers) that used an old addressing standard (which embedded the MAC address of the device into the IPv6 address) and Wi-Fi geolocation databases in order to accurately locate devices. Older IPv6 networking stacks were encouraged to use the MAC address of the network adapter as the least significant bits of the full IP address. This process has changed due to the glaring privacy concerns, but routers are rarely updated. Over the course of a year, the research team scanned and discovered 61M unupdated devices that encoded the MAC address into their externally-reachable IPv6 addresses.

Exploiting the low-entropy in MAC addresses on a router between the WAN, LAN, and wireless adaptors, the researchers developed tooling to link MACs from the same device. Combining this with the crowd-sourced Wi-Fi geolocation databases (e.g., WiGLE), the researchers were able to geolocate ~12M routers in over 100 countries with a median error of just 39 metres.

TAKEAWAY:

- ✓ **Networking and other embedded devices** are designed to have a long lifespan; in many cases longer than the vendor chooses to support them (or even exists). When coupled with the flat networking of IPv6, this can lead to some interesting security and privacy issues. Too many products and services are designed with the mindset that they will operate within a NATed network and not be reachable externally, and IPv6 is forcing this model to change. There will be more pain associated with that shift in the time to come.

Figure 1. A high-level diagram of the geolocation process from an IPv6 address to a ~39m location.



Device Tracking via Linux's New TCP Source Port Selection Algorithm

Authors: Moshe Kol, Amit Klein, and Yossi Gilad



[Citation](#)



[Paper](#)



[Code](#)

This research explored methods by which an attacker could remotely track a specific device across multiple networks. By inserting a small snippet of JavaScript code into the victim's browser, the researchers were able to uncover a randomised key that is used by modern Linux kernels to determine which TCP source ports will be used next. That key is static for the duration of the system's uptime, and across every container sharing the same kernel. This enables an attacker who is able to determine that key in (e.g., through an injected web-ad) to track that device across networks, through VPNs, etc.

The Linux kernel in versions 5.12-rc1-5.15.39 upgraded the algorithm for selecting the next port to use for a connection (to protect against DoS or TCP session hijacking attacks) and introduced this key-based permutation scheme. Unfortunately, this large key space is then used within a small array of perturbations, so JavaScript that uses WebRTC or AJAX can alternate between establishing a number of connections with the attacker's server (which will track the ports used), and a loop-back address, looking for collisions. Those colliding ports act as a unique fingerprint for the randomised key. On a modern Chrome browser, this attack takes between five and 15 seconds to perform, and works on a majority of ISPs. Networks that rewrite the ports used (typically NATs or some anonymising services like Tor) generally are not affected by this attack, but it's worth noting once more that IPv6 support bypasses those NAT protections.

TAKEAWAY:

- ✔ **While this particular issue has been addressed**, it is interesting to see a security feature resulting in a reduction of privacy. Seeing the results from IPv6-enabled networks highlights once more that there will be subtle issues that arise based on the legacy assumptions of the internet being a collection of IPv4 NATed networks.

Figure 2. A table showing the tracking results across various network providers. When the ISP supports IPv6, even NAT port rewriting can be bypassed.

Network	Port Rewriting?	Throttling?	IPv4 / IPv6
EduRoam	No	No	✓ / NA
University Guest	Yes	No	✗ / NA
Landline ISP 1	No	No	✓ / ✓
Landline ISP 2	No	No	✓ / NA
Landline ISP 3	No	No	✓ / NA
Landline ISP 4	No	IPv4 only	✓* / ✓
Cable ISP 1	Yes	No	✗ / NA
Cellular ISP 1	IPv4 only	IPv4 only	✗ / ✓
Cellular ISP 2	IPv4 only	IPv4 only	✗ / ✓
Cellular ISP 3	No	Yes	✓* / NA

* With slowed-down TCP SYN bursts.

zk-creds: Flexible Anonymous Credentials from zkSNARKs and Existing Identity Infrastructure

Authors: Michael Rosenberg, Jacob White, Christina Garman, and Ian Miers



[Citation](#)



[Paper](#)



[Code](#)

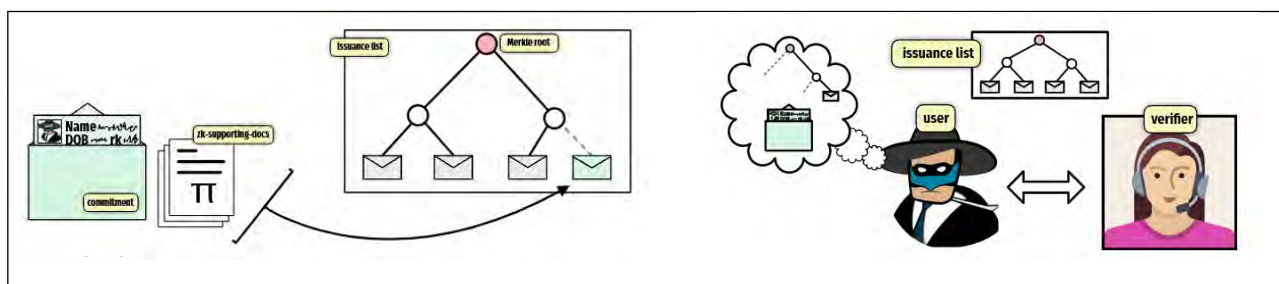
This research explored using zero-knowledge (ZK) proofs to build digital identities (ID) from trusted identity documents (i.e., passports) that can be used in flexible and privacy-preserving ways. The flexibility allows for specific attributes, or dynamic properties of ID attributes to be computed in a zero-knowledge manner, for example: that an ID has not expired, and that the user is older than a certain age. These dynamic properties do not need to be pre-determined at system creation time, a marked improvement over the current state-of-the-art infrastructure, and it allows for more than one-size-fits-all use cases. The privacy-preserving features allows for a proof to be verified that, e.g., the ID is from a trusted provider and the user is greater than a certain age without revealing the ID provider, the user's date of birth, or any other ID information (such as name).

While the mathematics behind ZK proofs are complex, the key innovations that underlie this work are the ability to link and blind proofs, letting the system dynamically combine specific elements of information – both about an identity and the overall system, without revealing additional information. Additionally, the system is flexible with respect to the deployment; it can be decentralised from the ID providers, or hosted by a trusted entity. There are a number of tuneable parameters that shift the computational burden between proof generation and verification, depending on the use cases.

TAKEAWAYS:

Figure 3. Two figures representing the two stages of zk-creds use: 1) A credential is committed to a public Merkle tree based on information from a trusted identity issuer in a ZK fashion, and 2) attributes of that identity are shared with a verifier by the user while protecting the specifics of the identity.

- ✓ **These primitives** can allow for capabilities where there is no non-digital analog: essentially creating a non-forgable 21+ (or other attribute) identity card that is tied to a trusted issuer that may not be tech savvy enough to run an anonymous ID protocol. While the paper highlights seamless and privacy-preserving ways to access adult content, these primitives could allow for anonymous, but bot-free, social networks.
- ✓ **The fact** that this system does not need any additional support from the trusted identity providers (e.g., passport issuing authorities) other than digitally-signed identity data, makes bootstrapping this system into digital environments much faster and easier. This is an interesting space to watch!



3 Years in China: A Tale of Building a REAL Full Speed Anti-Censorship Router

Author: Kaijern Lau

 [Citation](#)

 [Slides](#)

 [Video](#)

 [Code](#)

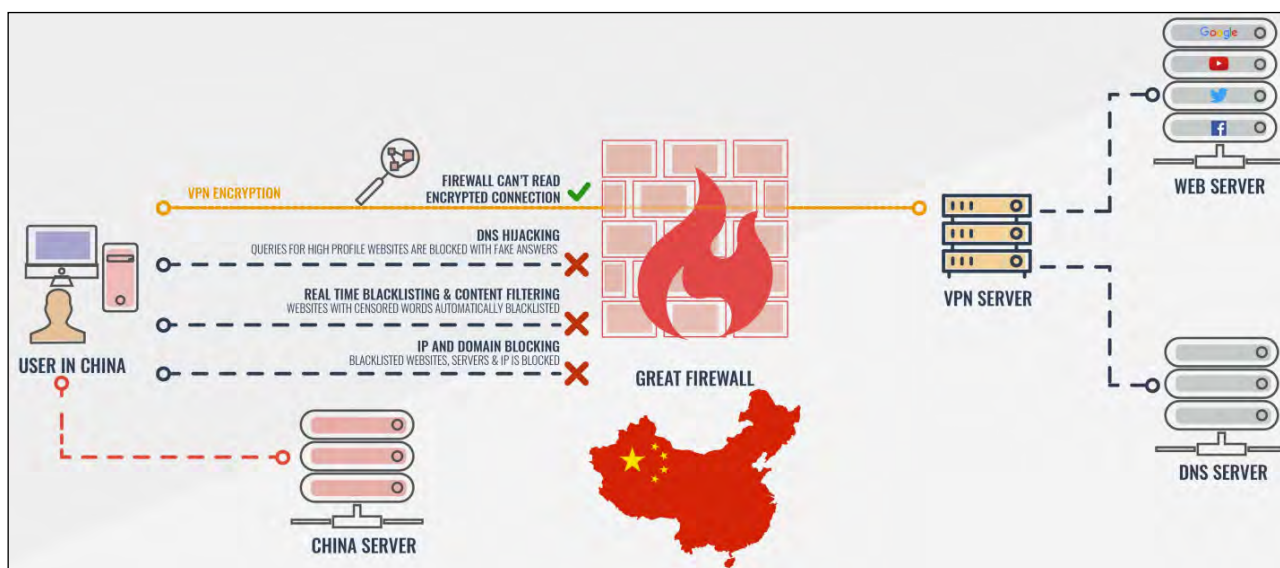
This talk is by a security researcher who was in China during the COVID-19 travel restrictions. During their unplanned three-year stay, they worked to understand the great firewall (GFW) of China that separates the domestic and international networks, as well as build easy-to-deploy bypass tools. While technically illegal, VPNs can be used to access the wider internet, but the GFW blocks access to many domestic services from external IPs, preventing access to deeply-entrenched services like Alipay or WeChat – without which it is difficult to live in China, especially when COVID-19 restrictions were in place.

The researcher explores how the GFW's capabilities change based on Party events, and how much human-in-the-loop is needed to tie online presence to physical entities, as well as offering a glimpse into the tools built to bypass the GFW restrictions. In building a plug-and-play OpenWRT fork, the researcher shows the tools, volunteers, and services designed to enable a seamless digital experience for both domestic and international applications.

TAKEAWAYS:

- ✓ **While the technology developed** is not groundbreaking in itself, the perspectives into how large an undertaking the GFW is, and how it can be shaped to the whims of the Party, are valuable. The human and compute capital deployed to tie online personas to the entity behind them, monitor and censor content, and adjust for party meetings is difficult to fully realise without these anecdotes.
- ✓ **The activity on the bypassing projects** (mostly in Chinese languages) highlights the scale of such [illegal] endeavours. That this level of maturity is tolerated shows the limits of the GFW.

Figure 4. A figure of how to both bypass great firewall restrictions while allowing for Chinese domestic application and service access.



Embedded [in]security

- ✓ *Embedded Threats: A Deep Dive into the Attack Surface and Security Implications of eSIM Technology*
- ✓ *RPMB, a secret place inside the eMMC*
- ✓ *Compromising Garmin's Sport Watches: A Deep Dive into GarminOS and its MonkeyC Virtual Machine*
- ✓ *The Impostor Among US(B): Off-Path Injection Attacks on USB Communications*
- ✓ *MagBackdoor: Beware of Your Loudspeaker as A Backdoor For Magnetic Injection Attacks*

*A sand dune of the Namib Desert in the Namib-Naukluft National Park.
Image by Johnstocker Photography..*

Embedded Threats: A Deep Dive into the Attack Surface and Security Implications of eSIM Technology

Authors: Markus Vevier



[Citation](#)



[Video](#)



[Code](#)

With eSIMs becoming more popular in mobile devices (phones, tablets, and laptops), this research explored the ecosystem of eSIM technologies, and identified some security vulnerabilities in them. Designed to be a drop-in replacement for traditional SIM cards, eSIMs provide the same level of deep integration with the radio and device, but in a form factor that is easier to deploy (and can be updated and controlled remotely). This researcher explored how eSIM profiles are deployed, the security measures in place to prevent malicious profiles, and what one could do if they had control over a profile.

Profiles are installed via a QR code, or a manually loaded URL that points to a web service that distributes the signed bundle of secrets, applications, and/or applets that will be provisioned into the eSIM hardware module. These profiles must be signed by a GSMA-approved certificate, but the research found that on many devices, this certificate validation was incorrect, and allowed for test-signed profiles to be installed (even without administrator access on Windows devices). Once a profile was installed, the research exposed the potential for privilege escalation as there were crashes observed in the system processes that interact with the eSIM module.

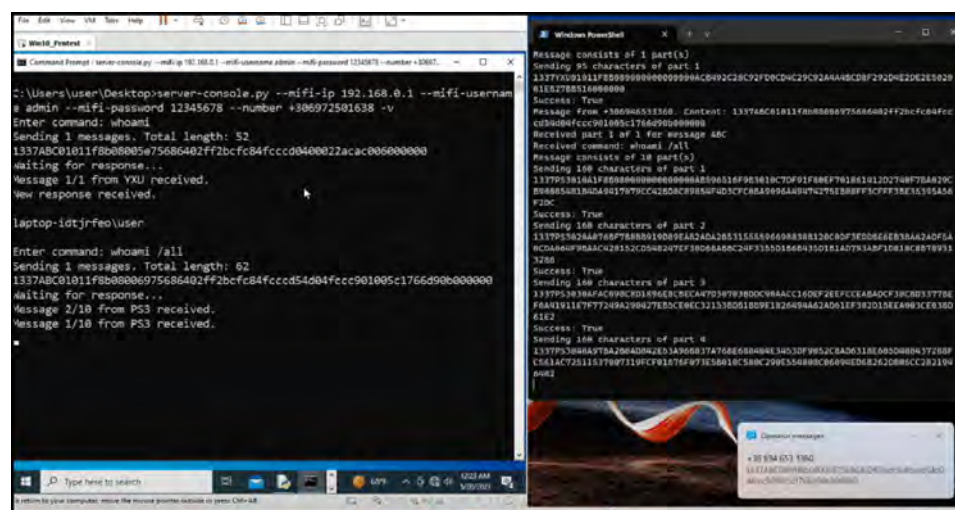
Finally, a proof-of-concept command-and-control system was developed that used SMS messages as an out-of-band communication path between the controller and the implant – running either on the target device, or within the radio hardware. Building on this research profile, a fuzzer developed for hardware SIM cards was ported to the eSIM modules for future exploration.

TAKEAWAY:



Finding large device manufacturers improperly allowing research/test keys to be used in practice is a glaring issue. It is very difficult to fix this long tail of vulnerability, which gives attackers control over an invisible part of the system. With the publication of these eSIM fuzzing tools and a large swath of hardware devices to test on, we expect more baseband vulnerabilities in the near future.

Figure 5. A screenshot showing the attacker computer (left) communicating with an implant (right) over an SMS channel using the malicious eSIM profile installed by a non-privileged user.



RPMB, a secret place inside the eMMC

Author: Sergio Prado



Citation



Blog

This blog post detailed the Replay Protected Memory Block (RPMB) partition found on many eMMC and NVMe (and other flash-based) storage devices. The RPMB partition contains a write-once unreadable section that can be set to an authentication key, and a read-only write counter that is incremented by the device upon each write. Building on those components allows for the host to authenticate data being read from the device and prevent replays.

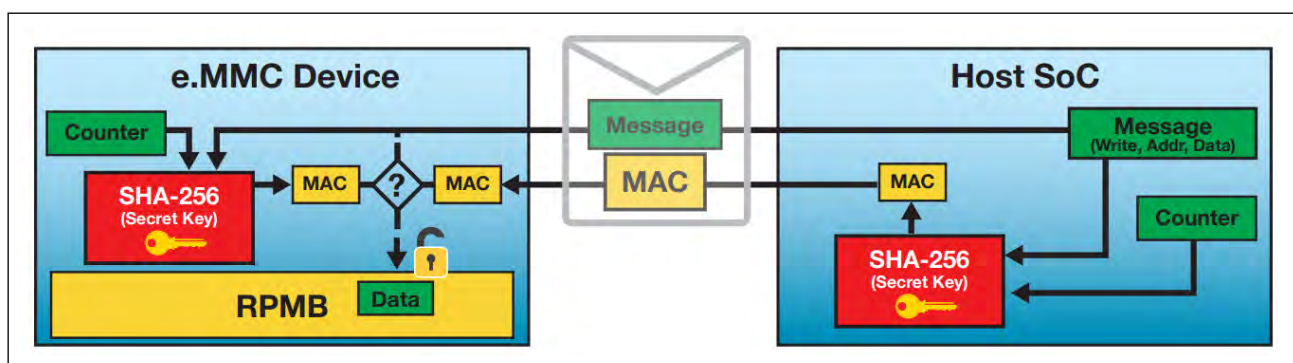
1. When reading data, the host sends a nonce, and receives the data requested and a HMAC that signs the data and nonce with the authentication key.
2. Authenticated writes include the counter value in the HMAC, so a replay of a properly-signed write will fail as the counter value will differ.

The RPMB has been standardised and support for it is built into modern Linux environments, allowing for devices to take ownership of eMMC devices by setting the authentication key, and then to authenticate reads and writes. The author notes that it is best practice for a trusted execution environment (TEE) to take ownership of the RPMB partition to prevent the authentication key from being exposed.

TAKEAWAY:

- ✓ **With these primitives available** on incredibly common and low-cost devices, and a push for more embedded CPUs to support trusted execution environments, there is less excuse for cheap IoT devices to be as insecure as they are. This primitive helps prevent malicious data injection or tampering, even against a moderately-skilled physical attacker. However, with the IoT space currently being so far behind in security, it will likely be many years before new devices use these types of protections.

Figure 6. A diagram showing how the RPMB partition on eMMC devices can be used to prevent replay attacks and authenticate data from the device to the host.



Compromising Garmin’s Sport Watches: A Deep Dive into GarminOS and its MonkeyC Virtual Machine

Author: Tao Sauvage

 [Citation](#)

 [Slides](#)

 [Video](#)

 [Blog](#)

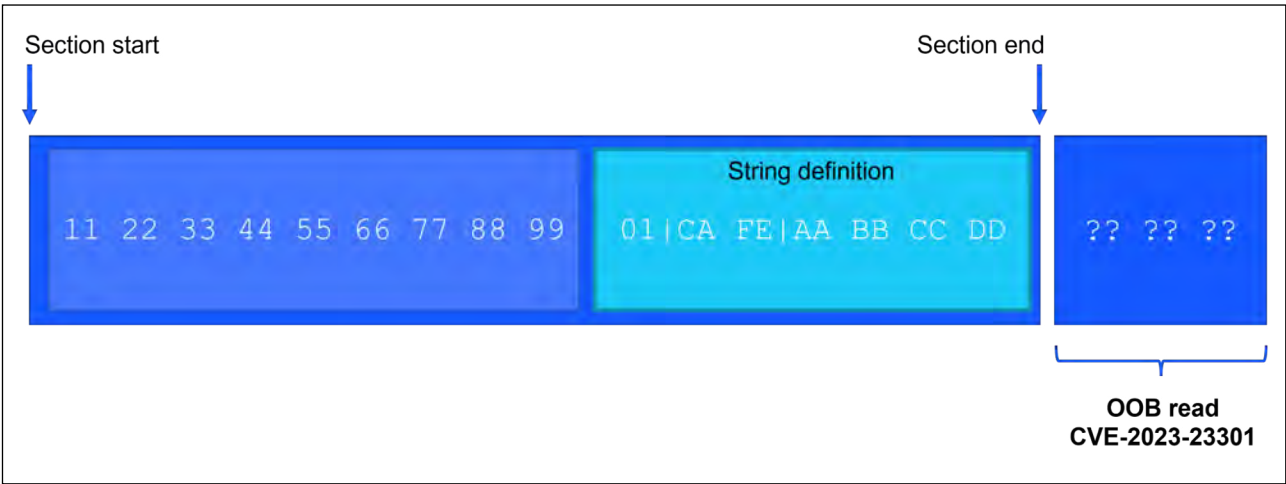
This research explored the custom software environment found on Garmin devices (i.e., smart watches and GPS fitness trackers). Unlike many other embedded devices, Garmin created their own custom operating system and virtual machine for 3rd party applications, which are written in a custom programming language called MonkeyC. Over the duration of the research, nine CVEs were assigned for multiple classes of vulnerabilities that allow for an application to escape the VM’s permission protections.

The research lays out the approach of exploring a novel software ecosystem, from examining forum posts for hints, to building on past research and extracting symbols from beta firmware builds and the SDK. A methodology of how to explore unknown/new targets is clearly explained and there are some great pointers for others who are interested in this type of embedded exploration.

TAKEAWAYS:

- ✓ **Many of these issues** have been present for a long time, reminding us that a custom system does not usually buy additional security (however, it may often delay the examination and discovery of issues). The design of the VM and use of a custom programming language can set up the platform for success on the battery life and responsiveness, but common security issues have persisted for many years.
- ✓ **There is a huge diversity** in the embedded consumer devices market, but finding a custom-written OS and stack that is also one of the most popular in use elevates the relevance of this type of exploration.

Figure 7. A figure showing how, even with sentinel values and lengths, a string can be constructed to read beyond the data section.



The Impostor Among US(B): Off-Path Injection Attacks on USB Communications

Authors: Robert Dumitru, Daniel Genkin, Andrew Wabnitz, and Yuval Yarom



[Citation](#)



[Paper](#)



[Code](#)

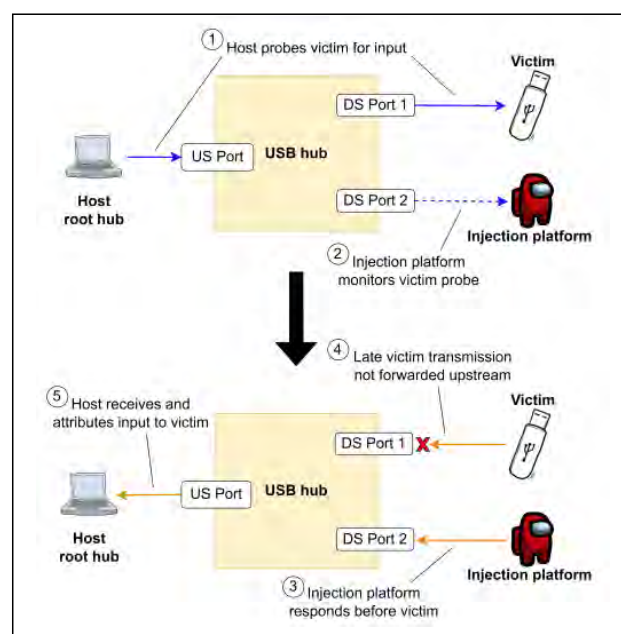
This research explored weaknesses in USB 1 and 2 that allow for a malicious device plugged into the same hub as a target device to man-in-the-middle MitM host-device interactions. While past work had shown that a malicious hub can trivially MitM the traffic flowing through it to the downstream devices, an off-path attack lets another downstream device inject traffic (that is interpreted as coming from the victim device) to the host. In response to the numerous AutoPlay-style attacks, many organisations have deployed policies that limit the allowed USB devices and vendors that are allowed by the host. These software defences can prevent a USB drive that also includes a keyboard profile to inject keystrokes – this research bypasses those defences at the hardware level.

USB 1 and 2 host-to-device transactions are broadcast, so all devices can listen in on communications from the computer to any other devices on the same root. This research explored how hubs handle multiple responses to a host request, and found that about half of the hubs tested (including those built into desktop motherboards) will send the first response received to the host. This allows for a faster malicious device's response to be sent to the host instead of the legitimate response. The paper concludes with a couple of example attacks built upon this primitive: a USB device that injects keystrokes that appear to the host to be coming from the allowed and expected USB keyboard; and a mass storage attack that replaces some contents of a live-booted Linux instance as it is loaded off of a legitimate USB drive.

TAKEAWAY:

- ✓ **While physical access to a computer** will almost always result in compromise, this research shows that more considerable defences are needed in order to prevent any USB device from activating a malicious sub-routine in firmware and bypassing all host-based checks and defences. While USB 3 solves some of these issues in being a directly-addressed protocol, Thunderbolt and other protocols that run over USB cables offer their own issues with DMA and bus-mastering.

Figure 8. A state diagram of the steps for the malicious device to listen for an opportunity to inject a response that would be interpreted by the host as coming from the victim device.



MagBackdoor: Beware of Your Loudspeaker as A Backdoor For Magnetic Injection Attacks

Authors: Tiantian Liu, Feng Lin, Zhangsen Wang, Chao Wang, Zhongjie Ba, Li Lu, Wenyao Xu, and Kui Ren



[Citation](#)



[Paper](#)



[Code](#)

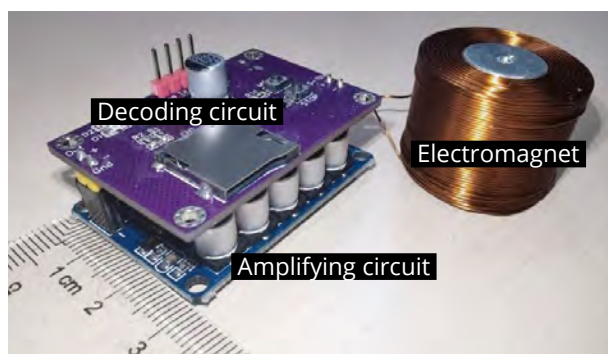
This research explored a new modality for voice assistant injection attacks (e.g., “Hey Alexa, unlock the front door”): magnetic fields. Exploiting the close proximity between speakers and microphones on modern devices, the researchers set out to explore if using an electromagnet to stimulate the speaker on a device could then create audio within the device sufficient to be registered by the microphone. Using a \$5 device that runs off of a USB-C power bank, they were able to achieve an above-90% success rate across a host of mobile devices, laptops and smart speakers at a range of 2.5cm (dropping to ~80% at 5.5cm).

While it is an early prototype, the researchers started with a weaker electromagnet and scaled up the power, showing that this could be achieved with a higher stand-off distance. The attack is relatively robust to the relative orientation of the attacker device and the target, and is simple: connecting the electromagnet to the audio output with the attacker command produces a magnetic field variation that is then converted to audio by the target speaker (and picked up by the nearby microphone).

TAKEAWAY:

- ✓ **While the practical applications of this attack are minimal**, it does highlight the externalities of ever more tightly-coupled components in modern devices. The relative ease of transmission of an audio signal to an electromagnet which is then converted to sound by the target device’s speaker means this could be scaled up in a more powerful, permanent installation where people may try to inject malicious voice commands as they walk past devices. While still very much a “Mission Impossible” attack, it is always nice to explore the art of the possible.

Figure 9. Two figures, (a) showing the scale of the attacker device with a 5.5cm range, and (b) showing the physical layout of the proposed attack.



(a) MagBackdoor prototype



(b) Attack scenario

*Wild fynbos growing in Table Mountain National Park, Cape Town.
Image by Yuri Arcurs.*

Issues at the operating system level

- ✓ *(Windows) Hello from the Other Side*
- ✓ *Every Signature is Broken: On the Insecurity of Microsoft Office's OOXML Signatures*
- ✓ *Dirty Bin Cache: A New Code Injection Poisoning Binary Translation Cache*
- ✓ *The Most Dangerous Codec in the World: Finding and Exploiting Vulnerabilities in H.264 Decoders*

(Windows) Hello from the Other Side

Author: Dirk-Jan Mollema

 [Citation](#)

 [Slides](#)

 [Code](#)

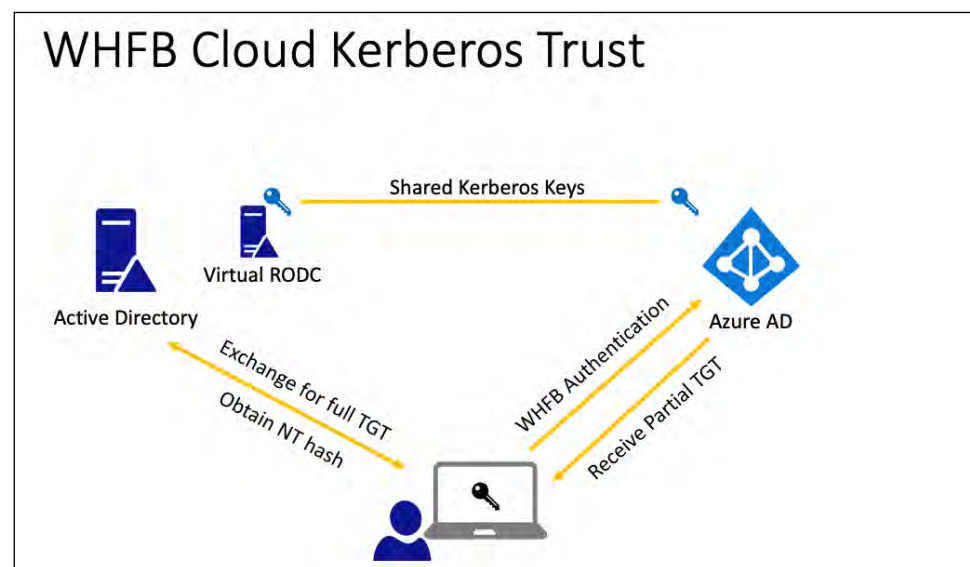
Windows Hello for Business is a passwordless authentication feature that uses a combination of device identity and biometrics (or PIN) to authenticate to Windows and (Azure) Active Directory. The researcher looked into the protocols and communication flows, discovering various vulnerabilities that could allow attackers to abuse Windows Hello to persist access to accounts, move laterally between identities and bypass Multi Factor Authentication.

Additional vulnerabilities were discovered with which attackers can bypass the hardware protection of secrets that allow the Windows Hello credentials to be used on different devices than they were provisioned on. One significant finding was that a Global Admin in Azure AD (with network connectivity to a Domain Controller) can both recover the NT hash of most synced users and obtain Domain Admin privileges.

TAKEAWAYS:

- ✓ **As new hybrid**, on-premise, and cloud authentication architectures are deployed within organisations, it will be important for teams to understand the attack surface, as well as understand new telemetry sources to detect and respond to compromise. The author published a number of tools along with the research, that can allow teams to validate their defence and find gaps before attackers do.
- ✓ **We may continue to see** an increase in attackers' leveraging of cloud authentication to persist and elevate privileges in both cloud and on-premise environments.

Figure 10. A diagram showing the flow of authentication between Azure AD and a local on-prem AD environment when using Hello. ▶



Every Signature is Broken: On the Insecurity of Microsoft Office's OOXML Signatures

Authors: Simon Rohlmann, Vladislav Mladenov, Christian Mainka, Daniel Hirschberger, and Jörg Schwenk



Citation



Paper



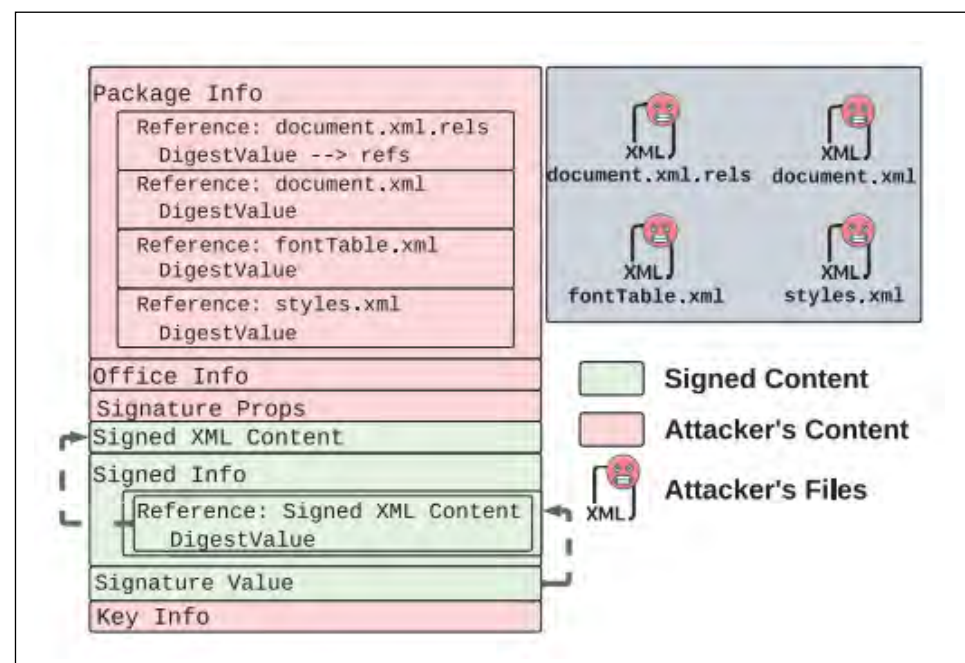
Code

Microsoft Office is one of the most widely used applications for office documents. For documents of prime importance, such as contracts and invoices, the content can be signed to guarantee authenticity and integrity. This paper applied a research methodology to understand both the document rendering flow as well as the signature validation flow. The team uncovered a number of gaps allowing attackers to circumvent trust assertions in these office documents. By using a rigorous approach, they found flaws in both signature and rendering flows. The signature flow is based on multiple relationships between protected content, and if any stage is not properly verified, attacks are possible. The researchers were able to create a document that passed signature tests and bypassed alerts even though the document contained arbitrary attacker content.

TAKEAWAYS:

- ✓ **Office documents** are based on nested XML relationships and authenticity and integrity continue to present a large attack surface. Undermining trust in signed documents may allow for an increase in new network initial access as well as fraud vectors to emerge.
- ✓ **Attackers and researchers** will continue to explore and exploit signature validation flaws. These can undermine the trust we place in both documents and binaries. This is certainly not the last we've seen of validation flaws, a trend we continue to monitor.

Figure 11. Universal Signature Forgery (USF). The attacker places a trusted Signed XML Content and references it in Signed Info. Only this content will be verified.



Dirty Bin Cache: A New Code Injection Poisoning Binary Translation Cache

Author: Koh Nakagawa



[Citation](#)



[Slides](#)



[Code](#)

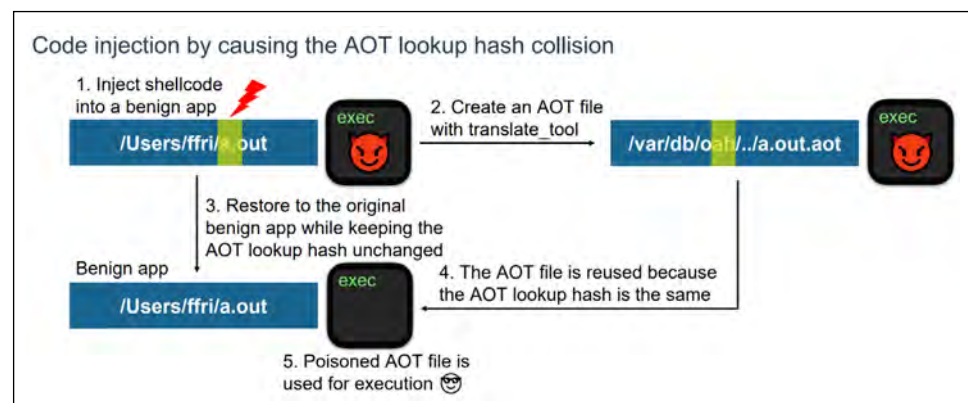
This work explores the security of applications that are converted from one CPU architecture to another. Both MacOS and Windows now support ARM-based systems, but have a large catalogue of software built for x64, necessitating the development of binary translators to allow for a legacy x64 application to run on an ARM computer. These translators can operate on-the-fly, or in ahead-of-time (AOT) modes – the latter model is the target of this research, specifically how the translated applications are cached and protected.

An analysis of the Rosetta 2 translator (for MacOS) determined that the cached binaries are protected by the kernel, but the way the cached binary is loaded when an x64 application is run is dependent on filesystem metadata that can be attacker-controlled. This allows an attacker to modify a target application, have Rosetta 2 cache the malicious application, then revert the target to its original state. When the application is run, it runs the malicious AOT cached binary, but forensically, there is no evidence of the changes. Additionally, the kernel prevents the cached binaries from being analysed.

TAKEAWAYS:

- ✓ **We expect** the relevance of this class of attacks to grow, then diminish. Cross-architecture applications are on the rise in the short term as the ARM market increases. Over time, more applications will release native ARM packages.
- ✓ **The shortcuts** highlighted in this work, i.e., code integrity-checks using hashes of metadata instead of application code, have not been fully addressed. Making point fixes to the specific metadata checked will not stem the issues that will crop up in these routines.
- ✓ **A similar attack** is described for Windows on ARM systems, which was determined by Microsoft to not be sufficiently concerning to warrant a rapid fix. Considering that code signing has been fundamentally broken on Windows for almost a decade, it is not likely that this will be addressed anytime soon.
- ✓ **The SIP protections** on the translated binary that prevent deep analysis of the potentially malicious version running on the system show the double-edged nature of some protections that add opacity.

Figure 12. A diagram of the code injection attack for MacOS's Rosetta 2 x64 to ARM translator.



The Most Dangerous Codec in the World: Finding and Exploiting Vulnerabilities in H.264 Decoders

Authors: Willy R. Vasquez, Stephen Checkoway, and Hovav Shacham



[Citation](#)



[Paper](#)



[Slides](#)



[Code](#)

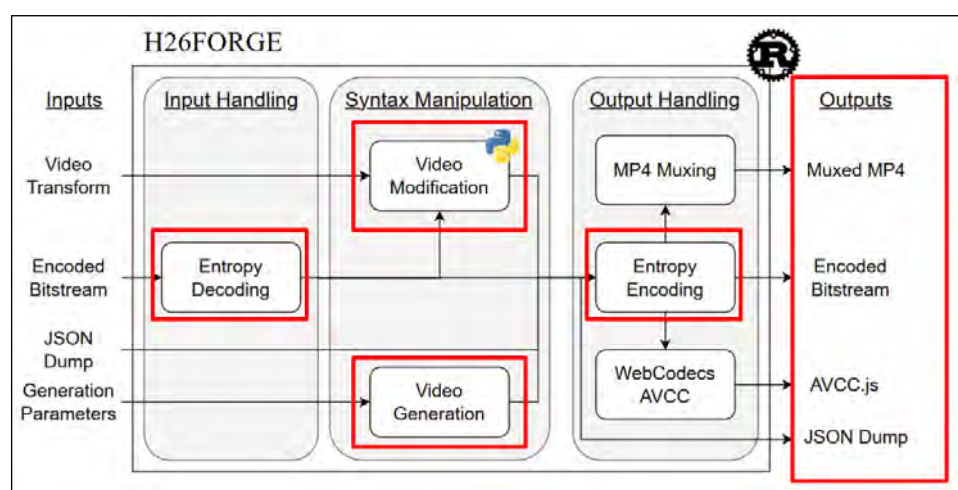
This work explored the complexity of parsing and rendering H.264 video (one of the most common formats in use today), and how that has parlayed into a number of high impact vulnerabilities. In digging deeper into these vulnerabilities, a lack of tooling in creating and modifying video files was discovered, prompting the development of H26FORGE. The tool suite allows for bit-specific modifications to a video stream, letting researchers target most aspects of the video decoding pipeline (see figure).

With this tooling, the researchers then explored and explained how a known, in-the-wild bug in Apple's H.264 decoder works, and how it would be possible to make the vulnerability more powerful with a chain of parsing issues. The approach of examining the large specification looking for data fields that are syntactically defined as a specific size, but semantically restricted to a smaller size, seems to be yielding promising results. Now that H26FORGE can target these, fuzzing and other vulnerability research techniques should greatly improve – resulting in better bug finding across the large ecosystem of decoders that run in hardware, operating system kernels, and browsers.

TAKEAWAY:

- ✓ **The large number of H.264 issues** reported or discovered in the wild before this tool was released indicates that there will be many more discovered with a more rigorous examination of the specification. Hoping a fuzzer hits the exact edge case that can trigger a crash has worked well enough, and with the ability to target the syntax/semantic edge cases directly, the number of bugs is sure to dramatically increase.

Figure 13. A block diagram of the different aspects of H.264 processing, and which aspects the H26FORGE tool can manipulate (highlighted).



*Sunrays filter through the rocks at Cederberg, South Africa.
Image by Sean Brookes.*

Nifty sundries

- ✓ *EverParse: Secure Binary Data Parsers for Everyone*
- ✓ *InfinityGauntlet: Expose Smartphone Fingerprint Authentication to Brute-force Attack*
- ✓ *It's (DOM) Clobbering Time: Attack Techniques, Prevalence, and Defenses*
- ✓ *Can you trust ChatGPT's package recommendations?*
- ✓ *Phoenix Domain Attack: Vulnerable Links in Domain Name Delegation and Revocation*
- ✓ *Man-in-the-Middle Attacks without Rogue AP: When WPA's Meet ICMP Redirects*

EverParse: Secure Binary Data Parsers for Everyone from zkSNARKs and Existing Identity Infrastructure

Author: Tahina Ramananandro



[Citation](#)



[Slides](#)



[Code](#)

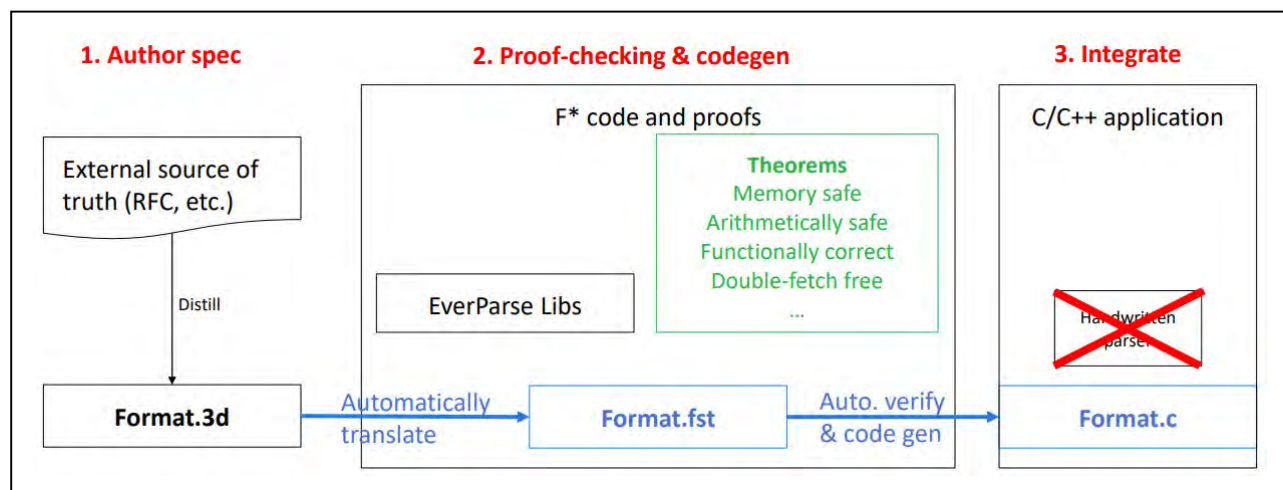
This keynote explored the vision EverParse and how that vision has been put into practice. EverParse is an open-source Microsoft toolchain to formalise on-the-wire protocols and build formally-verified parsers/serializers for them to remove unsafe and buggy hand-written code. While there has been a big shift to ProtoBufs and the like for safer (de)serialisation, there is a capability gap for handling pre-existing on-the-wire protocols – one that EverParse solves. Building a C-like structure format for the protocol lets developers quickly annotate existing C code with more validation and semantic information from the protocol. Those annotated C structures are then translated into a combination of F* and Low* for: verification for the absence of double-fetch bugs and memory/integer overflows. Finally, that verified code is extracted as a C library to replace the hand-written code.

EverParse has been used in production for all Hyper-V VMBUS code in Windows 10, 11, and Azure. The ~6K lines of specification is expressed as ~30K lines of C that have less than a 2% performance overhead, and when deployed the feedback from the security team is that their fuzzers immediately stopped finding crashes. TLS, QUIC, and ASN.1 have also been formalised and the resultant C parsers used.

TAKEAWAYS:

- ✓ **Seeing a project** move such a large portion of security-critical code from hand-written to formally-verified and automatically generated is impressive. That upon deployment the security team's fuzzing ceased to find parser crashes sells this approach. Letting the security team look for higher-level issues (e.g., logic bugs or authentication flaws) instead of parsing flaws should result in a more productive team and less weaknesses.
- ✓ **The ability** to convert from many commonly-used formats into verified C is a boon to many, and the open-source nature of EverParse means that there is almost no excuse to still be hand-writing parsers these days.

Figure 14. The three steps of using EverParse to remove unsafe, handwritten parsers from production code.



InfinityGauntlet: Brute-force Attack on Smartphone Fingerprint Authentication

Authors: Yu Chen, Yang Yu, and Lidong Zhai

 **Citation**

 **Paper**

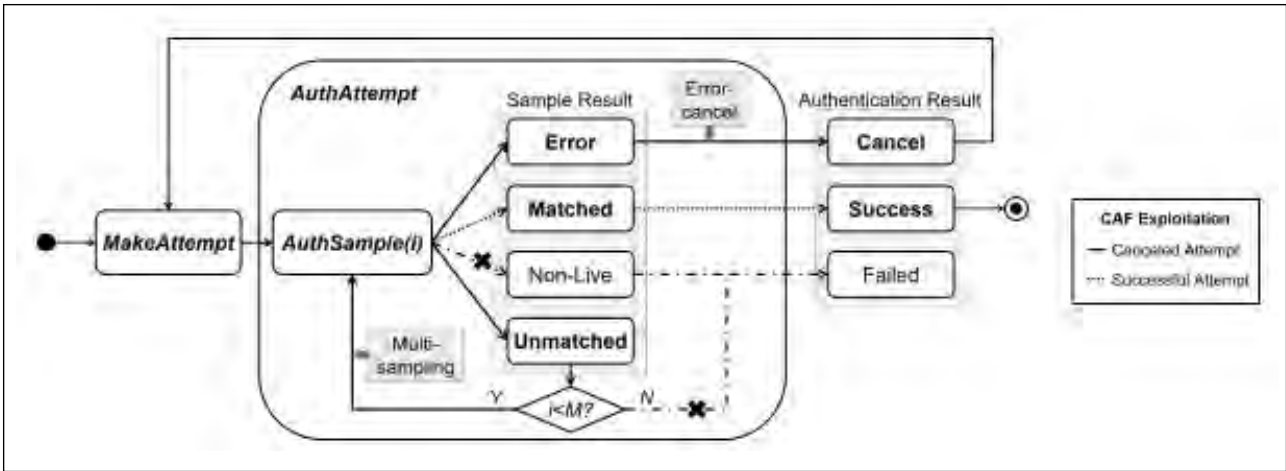
This research explored weaknesses in mobile device fingerprint authentication in the presence of a physical attack. Both iOS and Android devices permit a maximum number of biometric login attempts before either requiring a waiting period or using a different authentication scheme. Using their attack, these limitations can be either increased or removed, allowing for brute-forcing of biometric authentication. Assuming an attack has physical access to the device, and a fingerprint database, the attack consists of two stages: learning how each sensor represents the fingerprint data on the SPI bus, and attacking the attempt limiter.

The core of the limit bypass is shown in the figure below. When an authentication attempt is made, if the fingerprint collected matches the known fingerprint, it will immediately go to the success state. If it doesn't match, it will increment the failed attempt counter, and after the counter exceeds the maximum number of attempts it will limit fingerprint authentication. However, if there is an (e.g., hardware) error in obtaining or comparing the sample, the state machine will cancel the authentication process entirely, resetting the counter. This is designed to gracefully handle transient hardware issues, such as a corrupted checksum or SPI message. However, by injecting a purposeful invalid checksum every few attempts, the counter never exceeds the threshold and unlimited attempts are allowed. The researchers demonstrate this attack (and another variant) on a number of different Android devices from different vendors.

TAKEAWAY:

- ✔ **While the practical uses of this attack are limited**, it does highlight the possibility of contention between security and fault-tolerance. A system designed to gracefully handle occasional hardware issues can be subjugated to support the weakening of security. Often these are both considered good design practices, but their collective use can have nuanced issues crop up.

Figure 15. A state-diagram of how the fingerprint authentication process works and how it can be subverted by injecting errors.



It's (DOM) Clobbering Time: Attack Techniques, Prevalence, and Defenses

Authors: Soheil Khodayari and Giancarlo Pellegrino

 [Citation](#)

 [Paper](#)

 [Code](#)

 [Website](#)

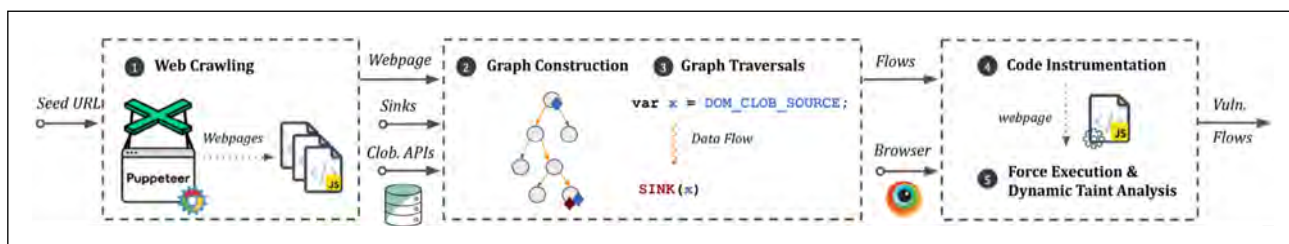
This research explored and categorised the attack class of Document Object Model (DOM) clobbering. DOM clobbering exploits the overlap between the HTML and JavaScript view of a page, possibly opening up sites that block JavaScript uploads to attack. For example, setting the iframe id to self can cause confusion when self is used as a JavaScript state variable (e.g., setting `window.location = self.location` can allow for an iframe to hijack the entire page). The researchers started with codifying and formalising the various variants of this attack class, checking browser impact, then building “TheThing” (see Figure below), a pipeline that would download websites and check for vulnerable flows, then verify those weaknesses in a browser.

In a survey of the top 5 000 websites by traffic, almost 10% were susceptible to DOM clobbering attacks, including big names such as GitHub and Vimeo. Finally, the researchers evaluated the existing defences and found that more than half of them were insufficient to the variety of variants, and that 10-13% of websites would break if more stringent defences were enabled.

TAKEAWAYS:

- ✓ **While checking for JavaScript or SQL** in user-submitted fields is top-of-mind for most web developers, the sheer volume of possibilities for these DOM clobbering attacks makes secure development much more difficult.
- ✓ **Even though existing protections** only provide partial coverage, building your sites with the knowledge of these mitigations allows for a seamless and smooth upward security curve as the tools get better and more restrictive.

Figure 16. A diagram of TheThing, a pipeline to gather empirical data about how popular websites may be vulnerable to different variants of DOM clobbering attacks.



Can you trust ChatGPT's package recommendations?

Authors: Bar Lanyado, Ortal Keizman, and Yair Divinsky



Citation



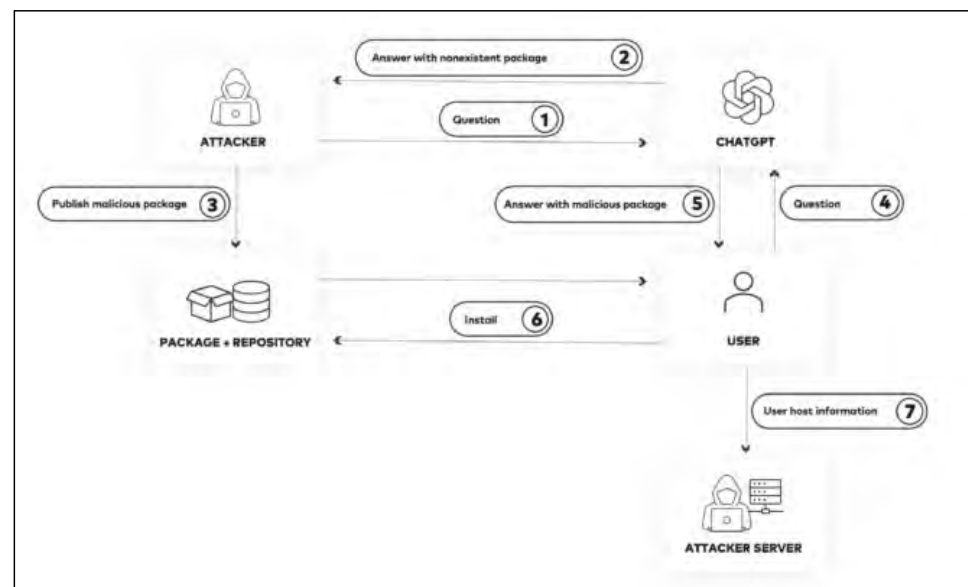
Blog

This blog post examined large language model (LLM) hallucinations, specifically when using generated code as a possible vector for spreading malicious software packages. When asking for package recommendations (both for Node.js and Python), or code that imports from third party packages, ChatGPT has been seen to create plausible sounding package names that don't exist. If an attacker can ask for a large number of recommendations, check for non-existent packages, and then create a malicious package with that name, when the LLM provides the same recommendation to another user, that user will likely install the package. In the authors' testing, they were able to generate package recommendations that, in over one third of the responses, contained a non-existent package name.

TAKEAWAY:

- ✓ **While this does not change** the underlying issues with software supply chains and packages that anyone can create, it changes the discoverability and spread of these packages. If there is even a chance that an LLM will hallucinate a specific package name again for other users (or those integrated into IDEs), then there is a decent opportunity for the malicious packages to be downloaded, installed and executed. Hopefully, as these LLMs are integrated into larger systems, guard rails will be put in place that prevent LLM output from directly turning into malicious code execution.

Figure 17. A figure outlining the attack steps to detect packages that ChatGPT may recommend that don't exist, then upload a malicious package with that name.



Phoenix Domain Attack: Vulnerable Links in Domain Name Delegation and Revocation

Authors: Xiang Li,
Baojun Liu, Xuesong
Bai, Mingming Zhang,
Qifan Zhang, Zhou Li,
Haixin Duan, and Qi Li



[Citation](#)



[Paper](#)



[Slides](#)

This research explored ways that an attacker whose domain has been sinkholed could continue to keep that domain resolving. While the authoritative DNS records are correctly removed, due to the extensive caching deployed across the resolvers, this research shows that it is possible to have a domain continue to resolve post-revocation. They outline two types of attacks that keep a domain in the cache past its official time-to-live. Due to vagueness in the specification for handling cache entries, a majority of DNS resolvers are vulnerable to edge cases in how they handle cache eviction and refreshes. One of the vulnerabilities presented is a TOCTOU vulnerability where a domain is about to expire from the cache. A DNS request is made for a sub-domain of the domain the attacker wants to preserve, timed such that the entry is still cached, but evicted during the request. By configuring the attacker's DNS server to answer slowly, the target resolver may act as if the domain is cached, but when it updates the entry, it resets the TTL instead as the entry was evicted. This allows attackers to keep their domain name present indefinitely, with a single request timed just right.

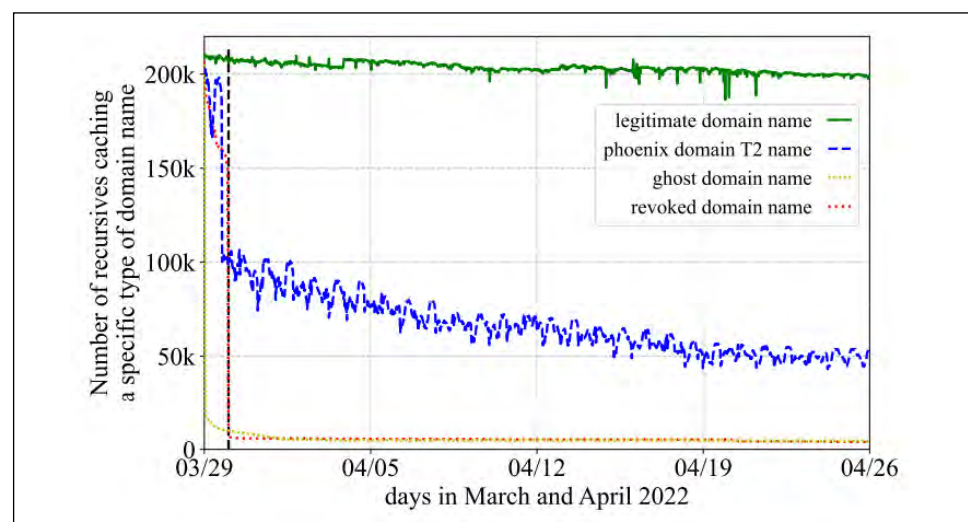
The second variant is a little more complicated, using some undefined aspect of how authority information for a domain is handled with sub-domains and expired caches for parent domains. In this variant, the attacker can successfully cache a sub-domain of the revoked domain, and repeat this process up to 127 times (the maximum subdomain depth), providing up to two years of additional life (see Figure below).

Both of these variants are being addressed by the resolver software, but as seen in previous DNS fix campaigns, rollout across the internet will likely take years to complete.

TAKEAWAYS:

- ✓ **While this research** highlights that even the most central and important systems on the internet today can have impactful flaws, federated distributed systems are complicated, especially with multiple differing implementations of the same specification. DNS has successfully weathered past security issues, and this should be no different.

Figure 16. A graph showing how a revoked domain name is still resolving from cache on 25% of resolvers a month later.



Man-in-the-Middle Attacks without Rogue AP: When WPAs Meet ICMP Redirects

Authors: Xuewei Feng,
Qi Li, Kun Sun, Yuxiang
Yang, and Ke Xu



[Citation](#)



[Paper](#)



[Website](#)

Figure 17. A state diagram showing the three stages of the MitM attack, allowing an attacker to intercept and modify Wi-Fi traffic on a WPA-protected network.

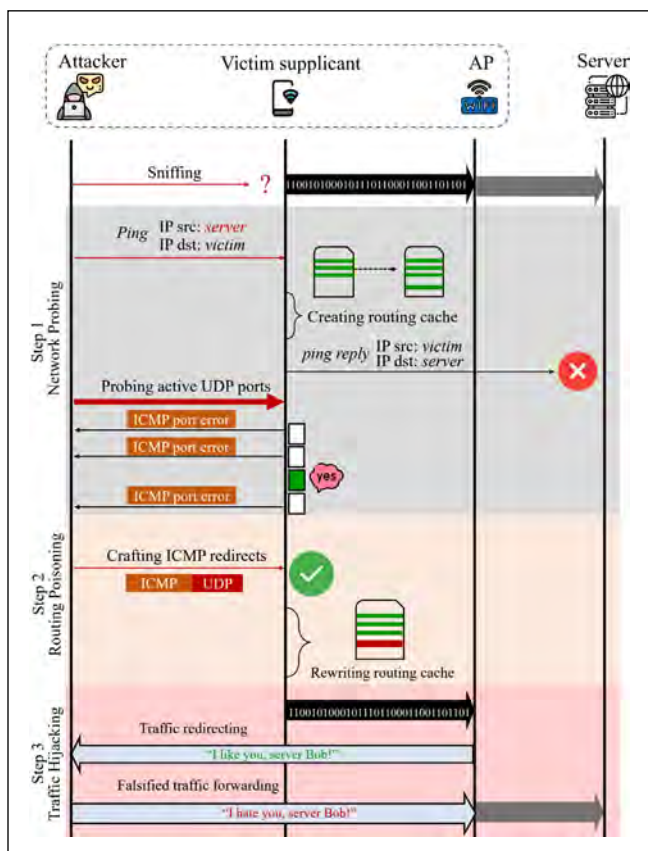
In this work, researchers explored ways to Man-in-the-Middle (MitM) other clients on the same protected Wi-Fi network. In WPA-protected networks, each client has a unique session key used to transmit between the client and the access point (AP), thus an attacker on the same network (e.g., sharing a library or coffee shop network) is unable to view or modify another client's traffic. This attack exploits a weakness in intra-network packet spoofing and a cross OSI-layer flaw to change a victim's routing table to route traffic to a target server through the attacker's client that has been encrypted with the attacker's session key.

First, the attacker sends the victim client an ICMP ping request with the source IP spoofed to appear as coming from the target server they want to MitM traffic to. As this is internal to the local network, it is not dropped by the routers, despite the source IP address being external, and the destination IP a private internal address. This ICMP request creates a routing entry on the victim's system to the target. Next the attacker probes for (or uses one of the commonly known) an open UDP port on the victim, such as 5353 for mDNS. Once this port has been identified, the attacker creates a hybrid ICMP and UDP redirect message and sends it to the victim (to the previously identified UDP port). The Linux and MacOS kernels vulnerable to this attack allow this packet to alter their routing tables because the UDP port references an open UDP socket, so it is assumed that it is valid. This redirect then sets the attacker's machine as the link-layer next-hop for communicating with the target server. Finally, all server-bound traffic from

the victim will be physically sent to the AP, but the AP will then decrypt and re-encrypt the traffic to the attacker's system, per the victim's routing instructions. This allows the attacker to view and alter all the traffic to and from the victim and the server (assuming there is no higher-level encryption such as TLS, etc.).

TAKEAWAY:

- ✓ **Many Wi-Fi networks** use WPA with a pre-shared key that is posted publicly – coffee shops, libraries, etc. This work shows that WPA alone is insufficient to guarantee every users' security to an attacker who is co-located. VPNs were a common recommendation for users of public Wi-Fi networks, and due to attacks like these, they remain a valuable defence.
- ✓ **The mitigation of tightening the restrictions** on accepting ICMP redirects where the sender and the transmitter differ seems like a best practice. These subtle cross-layer issues will keep cropping up while the mantra for network code is to be lenient in accepting malformed data.



Conclusion

This was a hefty issue! After a slow start to the year, the research community is heating up!

WE HIGHLIGHTED THREE UNIQUE THEMES THIS QUARTER:

1. Privacy in the modern era.
2. Embedded [in]security.
3. Issues at the operating system level.

Next quarter will include the “hacker summer camp” trio of conferences as well as the complete publication of all USENIX Security content – it should be another great one!

